React Frontend Application Development Using Vite

1. Introduction to React

React is a JavaScript library developed by Facebook (Meta) for building user interfaces, especially Single Page Applications (SPAs).

Key Characteristics

- Component-based architecture
- Reusable UI components
- Virtual DOM for high performance
- Declarative programming model

---

2. Prerequisites

Technical Knowledge

- HTML5
- CSS3
- JavaScript ES6+
- npm basics

Software Requirements

- Node.js (LTS)
- npm
- VS Code
- Modern browser

Verify installation:

```
node -v
npm -v
```

---

3. Creating a React Application Using Vite

```

```

```
npm create vite@latest app2
cd app2
npm install
npm run dev
```

---

4. Default Project Structure (Vite + React)

```
app/
├── node_modules/          # Dependencies (auto-generated)
├── public/                # Static assets
├── src/
│   ├── assets/            # Images / SVGs / media
│   ├── App.css            # App component styles
│   ├── App.jsx            # Main component
│   ├── main.jsx           # Entry point
│   └── index.css          # Global styles
├── index.html             # Root HTML file
├── vite.config.js         # Vite configuration
└── package.json           # Project metadata & dependencies
```

Detailed Project Structure Explanation

node_modules

- Contains all project dependencies
- Created automatically after running:

```
npm install
```

- Do not modify this folder

---

public

- Contains static files (images, fonts, audio, video)
- Files are served directly by Vite

---

src

Main source folder for application code.

assets

- Stores images, audio, video, SVGs

App.css

- CSS rules specific to App component

App.jsx

- Startup component
- Loaded when application starts

index.css

- Global CSS rules shared across components

main.jsx

- Entry point that renders the root component

---

pages

- User-created folder
- Contains page-level components

---

components

- Reusable shared UI components (Navbar, Footer)

---

services

- Backend communication logic (API calls)

.gitignore

- Files/folders excluded from Git repository

---

eslint.config.js

- ESLint configuration
- Helps detect syntax and quality issues

---

index.html

- Single HTML file (SPA)
- Contains `<div id="root">` host element
- Loads all React components

---

package.json

- Application configuration
- scripts, dependencies, devDependencies

---

vite.config.js

- Vite build and plugin configuration

Core Files Explanation

index.html

```
<!DOCTYPE html>
<html lang="en">
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

Key Point: Vite uses native ES Modules (`type="module"`).

---

main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

StrictMode Benefits

- Detects unsafe lifecycles
- Warns about deprecated APIs
- Detects unexpected side effects
- Development-only feature

createRoot

- React 18 concurrent rendering API

App.jsx

```
function App() {
  return (
    <div>
      <h1>Hello World!</h1>
    </div>
  );
}

export default App;
```

- JSX syntax
- PascalCase naming convention

---

5. Implementing Bootstrap in React

```
npm install bootstrap
```

Import in `main.jsx`:

```
// ☑ Bootstrap CSS (GLOBAL import)
import 'bootstrap/dist/css/bootstrap.min.css'

// Optional: Bootstrap JS (needed for navbar collapse, dropdowns)
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
```

---

6. Components in React

Components are reusable UI blocks written as JavaScript functions returning JSX.

Rules:

- Capitalized names
- Single parent element

---

7. Navbar Component

Used for global navigation. Placed in `src/components/Navbar.jsx`.

Uses `Link` from `react-router-dom` for SPA navigation.

---

8. Page Components

Located in `src/pages/`.

- Home.jsx
- About_Sunbeam.jsx
- Contact_Sunbeam.jsx

Each page corresponds to a route.

---

9. React Router

Install:

```
npm install react-router-dom
```

Core concepts:

- BrowserRouter
- Routes
- Route
- Link

---

10. Routing in App.jsx

Defines URL-to-component mapping and renders Navbar globally.

Example routes:

- `/`
- `/about`
- `/contact`

---

11. main.jsx

Entry point of the application. Renders App component into the DOM.

---

12. Running the Application

```
npm run dev
```

Open: http://localhost:5173

---

13. Application Flow

1. index.html loads
2. main.jsx initializes React
3. App.jsx handles routing
4. Components render dynamically
